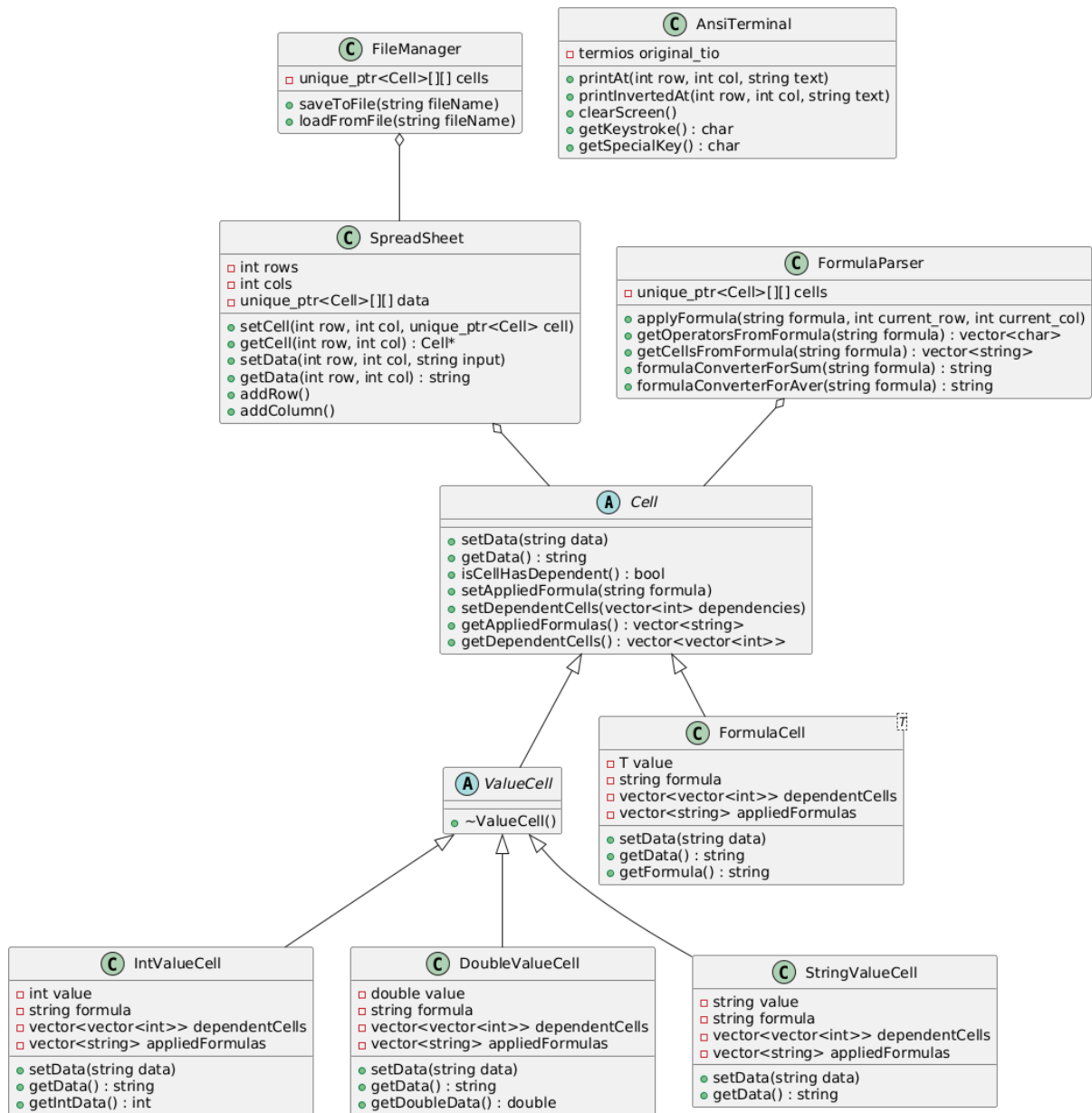


UML DIAGRAM:



SUM FUNCTION:

[illegible]

MAX FUNCTION:

	A	B	C	D	E	F	G	H	I	J
1	5	3	9	17						
2										
3										
4										
5										
6										
7										
8										
9										

Active Cell: Cell [D2]: =MAX(A1..D1)

MIN FUNCTION:

[illegible]

AVER FUNCTION:

[illegible]

[illegible]

SOME OPERATIONS EXAMPLES:

1-

[illegible][illegible]

[illegible]

3-

[illegible][illegible]

4-

[illegible][illegible]

Error Handling Examples:

1- Operation with number and string

Active Cell: Cell [C1]: =A1+B1

Active Cell: Cell [C1]: =A1+B1

```

A      B      C      D      E      F      G      H      I      J
1 testStr    15      Invalid argument while converting cell data: stod
gorkem@DESKTOP-345E5S4:/mnt/c/Users/Grkm/Desktop/oop_hw_2$ █
2
3
4
5
6
7
8
9

```

2- Try apply wrong formula format

Active Cell: Cell [C1]: =DUMPFORMULA

Active Cell: Cell [C1]: =DUMPFORMULA

	A	B	C	D	E	F	G	H	I	J
1	1	2	Invalid argument while applying the formula: stoi							
2	gorkem@DESKTOP-345ESS4:/mnt/c/Users/Grkm/Desktop/oop_hw_2\$									
3										
4										
5										
6										
7										
8										
9										

3- Operation with number and empty cell

Active Cell: Cell [C1]: =A1+B1

Active Cell: Cell [C1]: =A1+B1

```

1  10      Invalid argument while converting cell data: stod
gorkem@DESKTOP-345E5S4:/mnt/c/Users/Grkm/Desktop/oop_hw_2$ 
2
3
4
5
6
7
8
9

```

Implemented Features:

1. Cell Management

- **Cell Types (Cell class and its derived classes):**
 - **StringValueCell:** Stores string data in a cell.
 - **IntValueCell:** Stores integer data in a cell.
 - **DoubleValueCell:** Stores floating-point (double) data in a cell.
 - **FormulaCell:** Stores formulas and their computed results (implemented as a template class supporting both int and double types).
 - **Cell Operations:**
 - Write data to a cell (setData()).
 - Retrieve data from a cell (getData()).
 - Manage cell dependencies (add or retrieve dependencies on other cells).
 - Store and retrieve applied formulas for a cell.
-

2. Formula Management

- **Formula Parsing and Application (FormulaParser class):**
 - Extract operators from formulas (getOperatorsFromFormula()).
 - Extract cell references from formulas (getCellsFromFormula()).
 - Determine row and column indices from cell references (find_row_index(), find_column_index()).
 - Parse and compute formula results (applyFormula()).
 - Convert specific types of formulas:
 - **SUM:** Calculates the sum of a range of cells (formulaConverterForSum()).
 - **AVER:** Calculates the average of a range of cells (formulaConverterForAver()).
 - **MAX:** Finds the maximum value in a range of cells (formulaConverterForMax()).
 - **MIN:** Finds the minimum value in a range of cells (formulaConverterForMin()).
 - **STDDEV:** Calculates the standard deviation of a range of cells (formulaConverterForSTDDEV()).

3. Spreadsheet Management

- **SpreadSheet Class:**
 - Represents the spreadsheet as a 2D array of cells.
 - Dynamically resize the spreadsheet by adding rows (`addRow()`) or columns (`addColumn()`).
 - Write data to a specific cell (`setData()`).
 - Read data from a specific cell (`getData()`).
 - Access all cells in the spreadsheet (`getAllCells()`).

4. File Management

- **FileManager Class:**
 - Save the spreadsheet data to a CSV file (`saveToFile()`):
 - Writes cell values to a file in CSV format.
 - Load the spreadsheet data from a CSV file (`loadFromFile()`):
 - Reads cell values from a CSV file and populates the spreadsheet.

5. User Input and Interaction

- **Main Program (main.cpp):**
 - Real-time user input handling:
 - Navigate between cells using arrow keys (U, D, L, R).
 - Enter data into cells.
 - Detect and apply formulas (inputs starting with = are treated as formulas).
 - Display the active cell and user input in the terminal.
 - Save the spreadsheet to a file using the I key.
 - Load the spreadsheet from a file using the P key.
 - Exit the program using the q key.

6. Data Type Detection

- **Data Type Validation:**
 - Check if an input value is an integer (isInteger()).
 - Check if an input value is a double (isDouble()).
-

7. Dynamic Cell Updates

- **Dependent Cells:**
 - Manage dependencies between cells (if a cell depends on another, changes in one cell automatically update the dependent cells).
 - Recalculate formulas and update dependent cells dynamically.
-

8. Error Handling

- **Invalid Input Handling:**
 - Detect invalid data types and throw appropriate errors (e.g., stoi() or stod() exceptions).
 - Handle formula errors and display error messages to the user.
-

9. Performance and Optimization

- **Dynamic Memory Management:**
 - Use of unique_ptr to ensure proper memory management and prevent memory leaks.
 - Cells are dynamically created and managed as needed.

User Manual:

How to Run:

1- Compile the program:

```
g++ -o oop_hw_2 main.cpp AnsiTerminal.cpp FormulaParser.cpp  
FileManager.cpp Cell.cpp ValueCell.cpp StringValueCell.cpp IntValueCell.cpp  
DoubleValueCell.cpp SpreadSheet.cpp
```

2- Run the program:

```
./oop_hw_2
```

Keyboard Shortcuts

Key	Action
Arrow Keys	Navigate between cells (Up, Down, Left, Right).
I	Save the current spreadsheet to a file (e.g., <code>output.csv</code>).
q	Quit the program.
J (Enter)	Save entered data or apply a formula to the selected cell.
Backspace	Remove the last character of the input.
p	Load the spreadsheet from a file (e.g., <code>test_spreadsheet.csv</code>).

How to Use:

1. Navigating the Grid

- Use the arrow keys to move between cells:
 - o Up (U): Moves to the cell above.
 - o Down (D): Moves to the cell below.
 - o Left (L): Moves to the cell on the left.
 - o Right (R): Moves to the cell on the right.
- If you navigate beyond the current boundaries:
 - o New Row: Automatically added when moving beyond the last row.
 - o New Column: Automatically added when moving beyond the last column.

2. Editing Cells

- **Entering Values:**

1. Type a value directly into the current cell.
2. Press J (Enter) to save the value in the cell.

- **Deleting Values:**

- o Move to the desired cell.
- o Press Backspace until the value is cleared.
- o Press ENTER to confirm the deletion.

3. Using Formulas

Formulas start with an equal sign (=). Supported formulas include:

Supported Formula Functions		
Function	Description	Example
SUM	Adds values in a range of cells.	=SUM(A1..A5)
AVER	Calculates the average of a range of cells.	=AVER(B1..B3)
MAX	Finds the maximum value in a range of cells.	=MAX(C1..C10)
MIN	Finds the minimum value in a range of cells.	=MIN(D1..D10)
STDDEV	Calculates the standard deviation of a range.	=STDDEV(E1..E5)

4. Saving Data

- Press I to save the current spreadsheet to output.csv.
- The program overwrites the file with the latest data.

5. Loading Data

- Press P to load data from output.csv file.

AI-Based Assistance:

- 1- I used REGEX with the help of ChatGPT.
- 2- I sought help from ChatGPT to improve code readability and make some enhancements.
- 3- I used ChatGPT's assistance to create the UML diagram.
- 4- I received help from ChatGPT while preparing this PDF.
- 5- I got help from ChatGPT to set up the template structure in the FormulaCell.h file, especially regarding the usage of typeid functions.

The program's shortcomings:

- 1- The user interface in my program is slightly different from the interface of the VisiCalc program.
- 2- I cannot display the formula in the data section of cells where the result is given by a formula.